

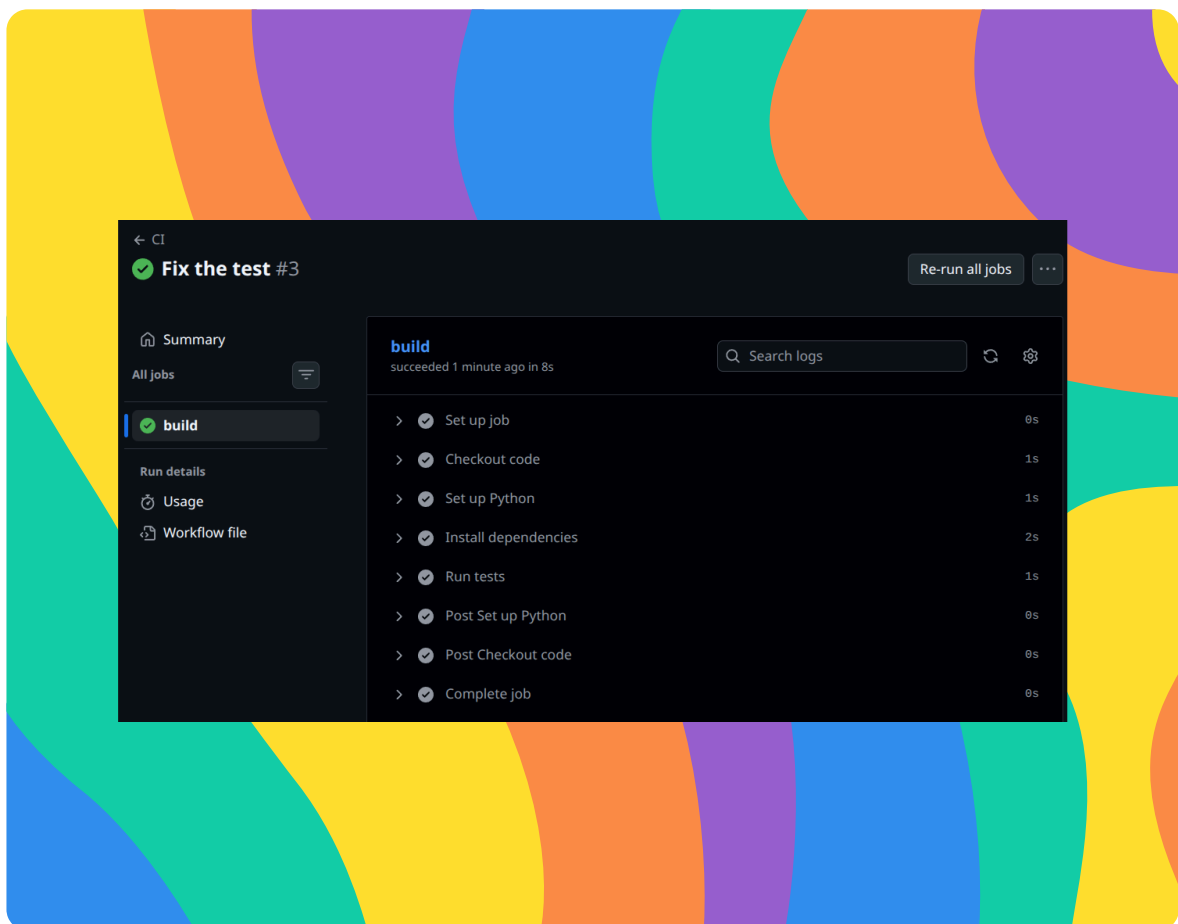


nextwork.org

Your First GitHub Actions AI workflow



Ahmed Tetteh





Introducing Today's Project!

In this project, I'm going to build a CI pipeline that catches bugs before they are deployed. CI/CD is a set of practices that revolves around the automation of software development lifecycle. In CI, developers frequently merging code changes into a central repository, while CD focuses on automating the release of those changes to users. GitHub Actions helps me to use a CI/CD pipeline on GitHub while collaborating with other developers to catch bugs quickly, before they become a problem.

Key tools and concepts

The key tools I used include GitHub Actions, Cursor, Python and pytest. Key concepts I learnt include how to create test scripts for errors in a CI workflow using GitHub actions, and building GitHub Artifacts.

Challenges and wins

This project took me approximately 2hrs. The most challenging part was writing the test and CI workflow scripts.

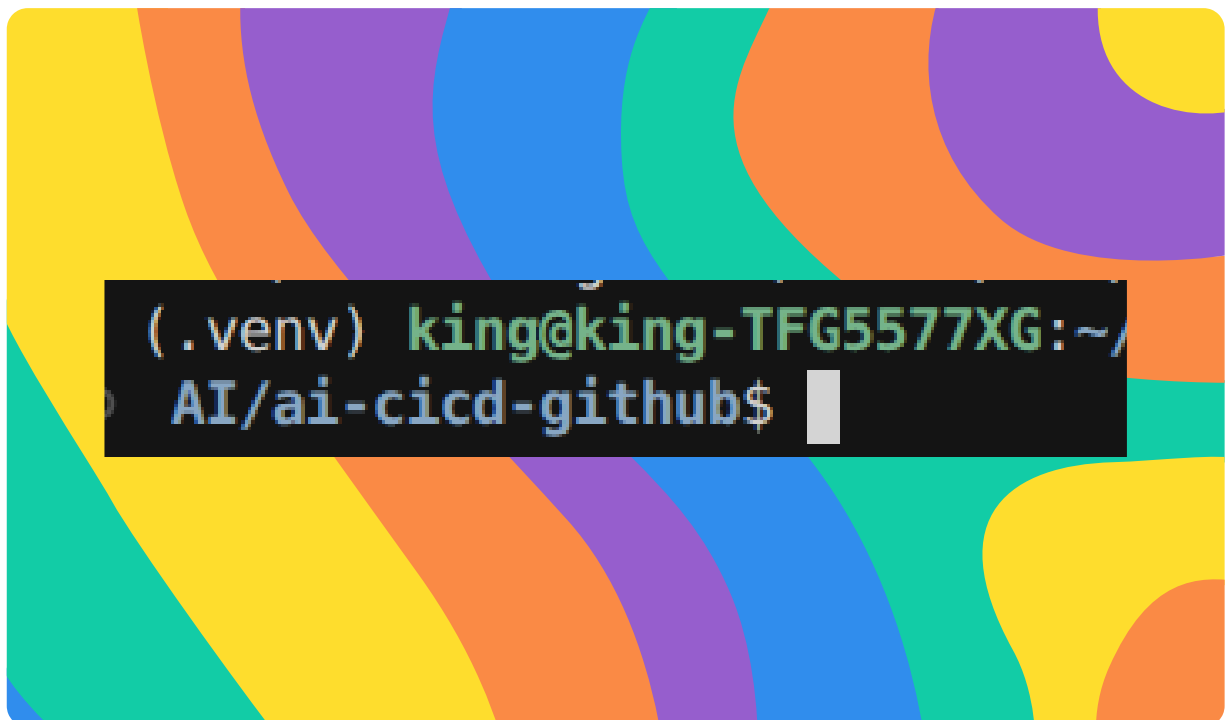
Why I did this project

I did this project today to learn how to developers use GitHub actions to accelerate their work in a team setting. Another skill I want to learn is how to deploy using GitHub actions.



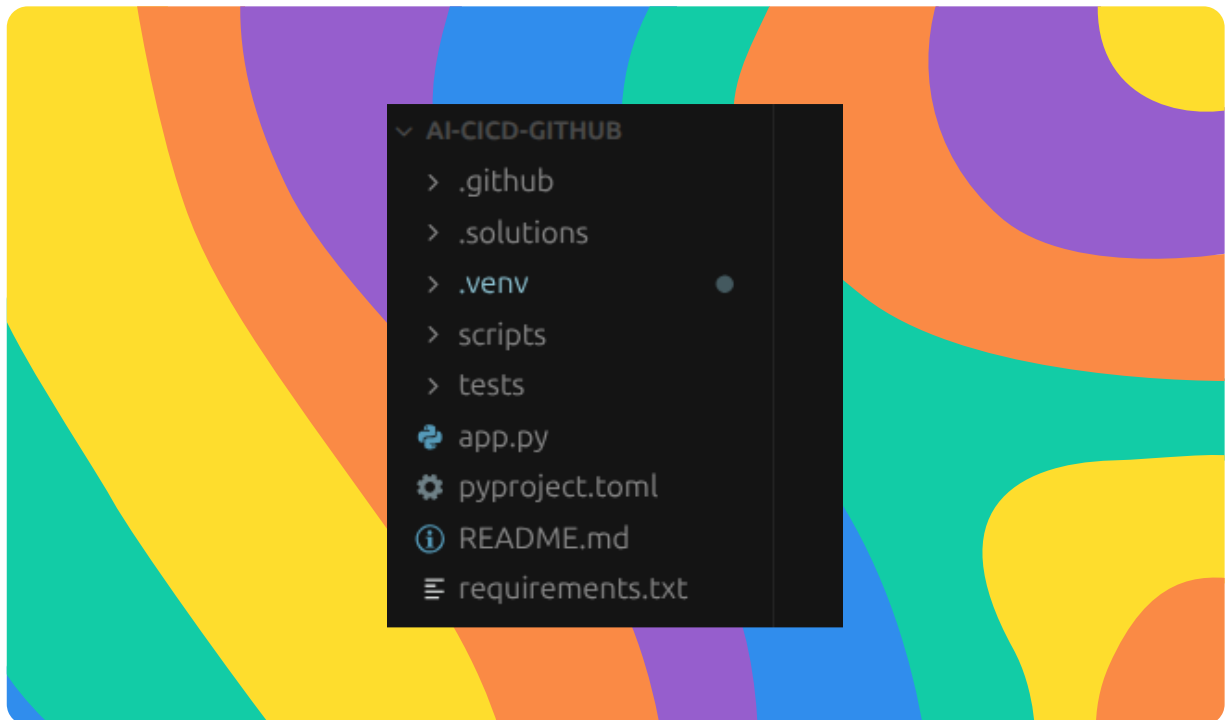
Setting Up the Python Environment

In this step, I'm setting up a development environment by forking a repo, creating a virtual environment in Cursor and installing any dependencies I might need for the project.



Activating the virtual environment

I activated my venv by "source venv/bin/activate ". The (venv) prompt means that python should install all the packages inside this virtual environment, not the python global one.



Exploring the project structure

I found key files including app.py, tests/, requirements.txt, pyproject.toml, and .github. The .github folder contains all my workflows for the CI/CD pipeline.



Writing and Testing a Python Function

In this step, I'm writing a test code to catch bugs. Tests are important because they ensure that the code behaves as expected (TDD).

```
17  
18  
19 def multiply(a: int, b: int) -> int:  
20     """Multiply two numbers."""  
21     return a * b  
22
```

Creating the multiply function

I wrote a multiply function that multiplies two integer values. Type hints help because they inform developers about the expected data type for a variable or function in Python.



```
(.venv) king@king-TF65577XG:~/Desktop/Personal Lab /DevOps & AI/ai-cicd-github
$ pytest -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0 -- /home/king/Desktop/Personal Lab /DevOps & AI/ai-cicd-github/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/king/Desktop/Personal Lab /DevOps & AI/ai-cicd-github
configfile: pyproject.toml
testpaths: tests
collected 6 items

tests/test_app.py::TestMath::test_add_positive PASSED [ 16%]
tests/test_app.py::TestMath::test_add_negative PASSED [ 33%]
tests/test_app.py::TestStrings::test_reverse PASSED [ 50%]
tests/test_app.py::TestStrings::test_is_even PASSED [ 66%]
tests/test_app.py::TestMultiply::test_multiply_positive PASSED [ 83%]
tests/test_app.py::TestMultiply::test_multiply_negative PASSED [100%]

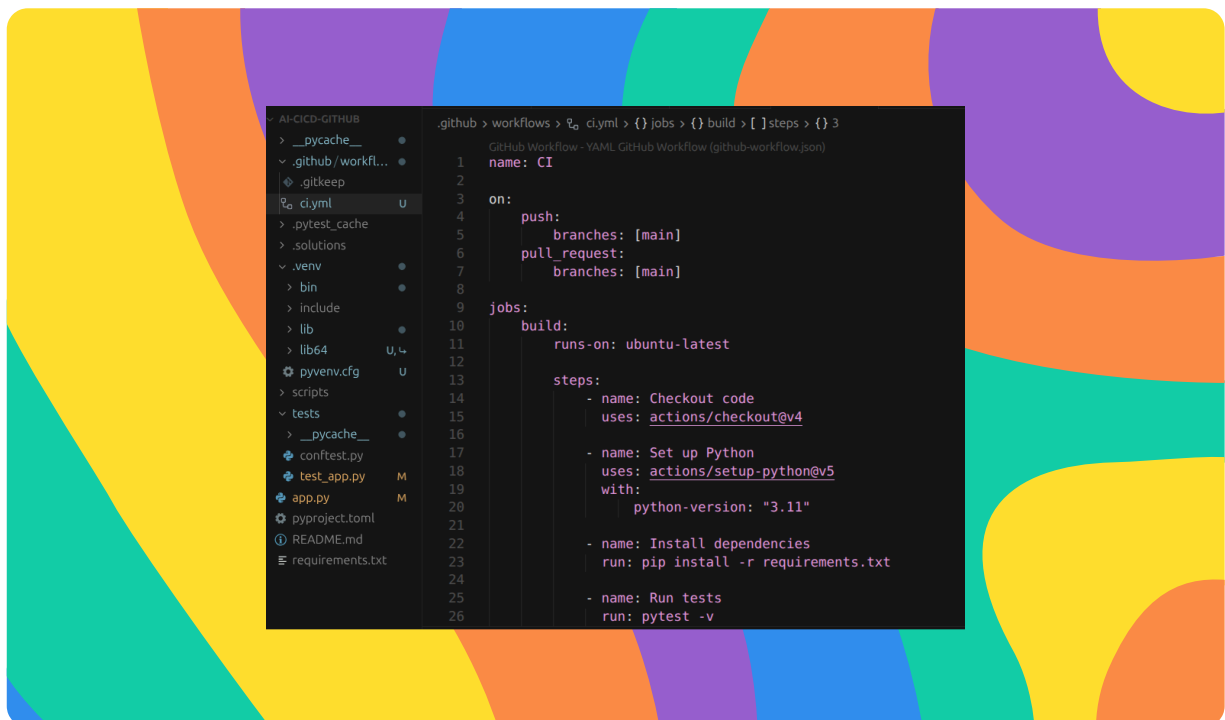
===== 6 passed in 0.01s =====
```

Running and verifying tests

I verified my tests by running "pytest -v" The output showed all 6 tests passed after running the functions, with the names of each test and their results.

Building a CI Pipeline with GitHub Actions

In this step, I'm creating a CI workflow file using GitHub actions. CI helps because it automates my tests upon each push to the github project repository.



Configuring the workflow

I configured the workflow to run the CI workflow on a push or pull request to the main branch. The steps include running the on(Triggers for the workflow), runs-on (OS type) and steps(commands to run on the server for each workflow).



Ahmed Tetteh

NextWork Student

nextwork.org

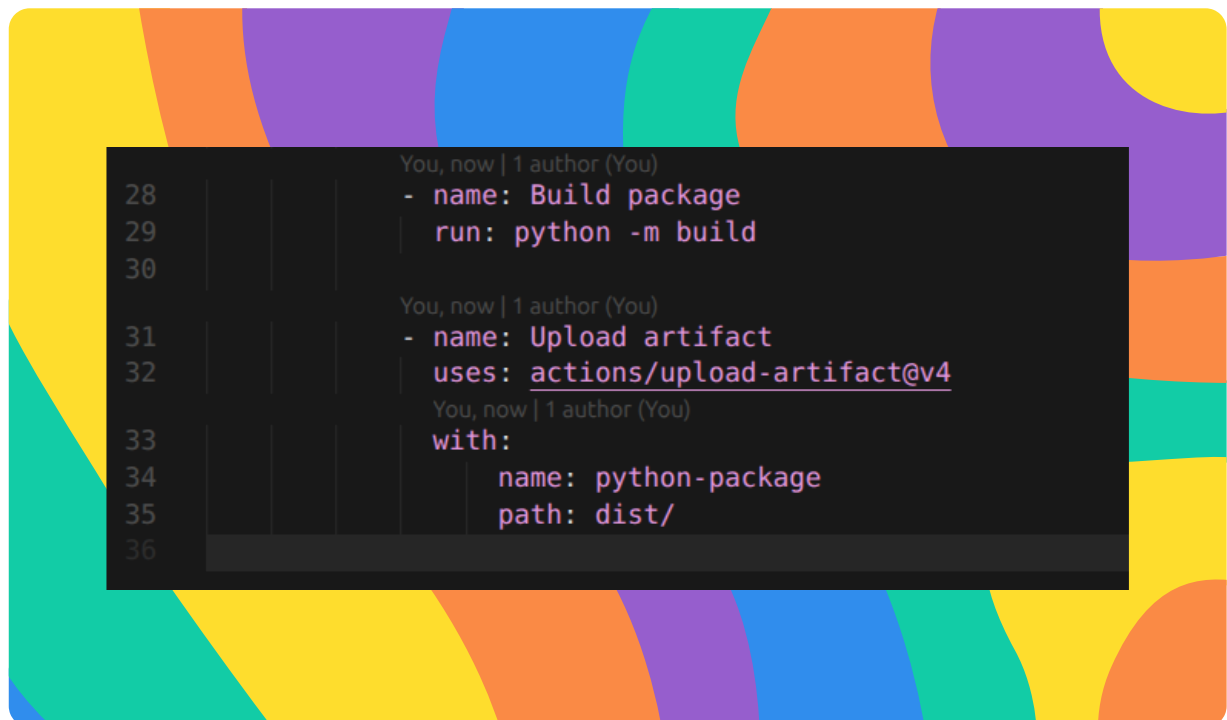
Testing the pipeline

CI caught the bug when I intentionally introduced an error to the test file. I fixed it by ensuring my assertion statements are true(in the test file). This shows CI helps because it catches the bugs early during uploads or pushes before it affects user experience.



Packaging Code with CI Artifacts

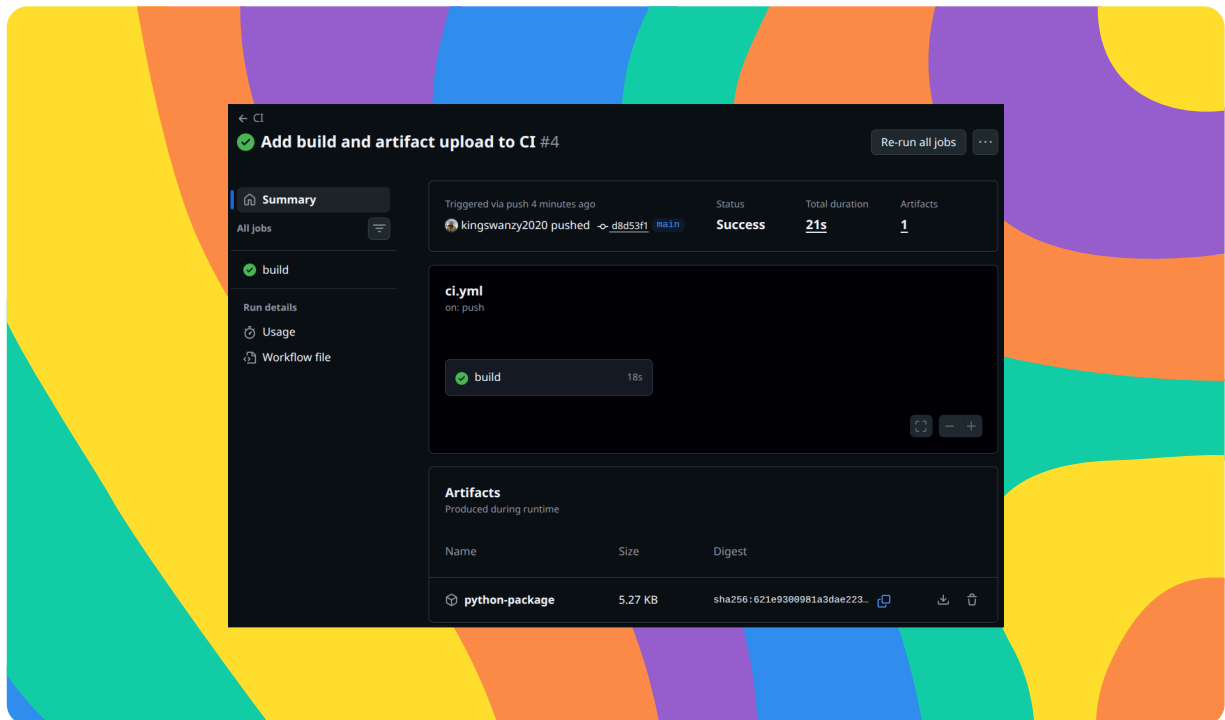
I'm adding a build step to my CI workflow for my python code to become easily distributable.



```
28 |         - name: Build package
29 |           run: python -m build
30 |
31 |         - name: Upload artifact
32 |           uses: actions/upload-artifact@v4
33 |           with:
34 |             name: python-package
35 |             path: dist/
36 |
```

Adding build and upload steps

I added a build step that packages my python code into two parts (.whl and .tar.gz file) and uploads as a single GitHub artifact for downloading and sharing. Artifacts are useful because they are they build outputs of workflows, test results or packaged code that can be deployed.



Downloading the packaged artifact

I downloaded my package from the artifacts section on my GitHub actions console. This means my CI pipeline can now build GitHub artifacts that can easily be downloaded and deployed anywhere using a command like "pip install your-package.whl".



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

